Software Design Document: NLP-Powered Text-to-SQL Interface

Version 1.0 | Implementer: Antigravity coding agent | Context: NLP AAT, 7th semester, B.M.S. College of Engineering

0. How to Use This Document (read first)

- Read the whole document before writing any code.
- Implement in the phases of Section 14. After each phase, run the checks in Section 15, report the actual outputs, and STOP for the human to confirm before the next phase.
- MUST = mandatory. SHOULD = strongly preferred. MUST NOT = forbidden.
- If anything is ambiguous, ask. Do not invent datasets, thresholds or behaviour not stated here.
- The human will read and explain every file in a viva. Code MUST be simple, commented, and readable by a student. Avoid clever one-liners and unnecessary abstractions.
- Build and test ONE module at a time, on its own, before connecting it to the pipeline.

1. Project Overview

Goal: a system where a user types a question in plain English about a college database, and the system returns the answer by generating and safely running a SQL query.

Why NLP matters here: this is graded as an NLP project, not an API wrapper. The LLM is only the final generator. The understanding of the question is done by our own NLP front-end: linguistic analysis, rule-based extraction, schema linking with sentence embeddings, value linking with fuzzy matching, and semantic retrieval of similar solved examples.

Running example used throughout:

> Average marks of pupils from Mangalore in dbms having attendance above 75

Problems hidden in it: pupils is not a table name (table is students); Mangalore is stored as Mangaluru; dbms is stored as DBMS; average and above 75 are hidden instructions.

Success criteria:

1. End-to-end pipeline works from a Streamlit UI.

2. Every NLP module is independently testable and its intermediate output is visible in the UI.
3. Evaluation on a 15-question test set reports execution accuracy, linking accuracy, and a no-NLP baseline comparison.
4. Destructive SQL can never run.

2. Scope

In scope: one SQLite college database; English questions; read-only SELECT queries; the NLP front-end; Streamlit UI; evaluation scripts.

Out of scope (MUST NOT build): fine-tuning any model, training any classifier, T5, multi-turn chat, authentication, deployment, multiple databases, write queries, a query-type classifier, a multi-version ablation study.

3. Tech Stack

Purpose	Choice
Language	Python 3.10+
Database	SQLite via the sqlite3 standard library
Linguistic analysis	spaCy with en_core_web_sm
Embeddings	sentence-transformers, model all-MiniLM-L6-v2
Fuzzy matching	rapidfuzz
LLM	Gemini through the google-genai SDK. Model name read from env GEMINI_MODEL (default gemini-2.5-flash; the human will confirm the current name in Google AI Studio)
UI	Streamlit
Misc	pandas, numpy, python-dotenv, pytest

Notes: use a virtual environment. If torch installation fails on Windows, install the CPU build. The API key lives only in .env and MUST NOT be printed, logged or committed.

4. Architecture

```
User question
     |
     v
[1 Preprocess/analyse]  tokens, lemmas, POS, noun chunks
     v
[2 Rule extraction]     aggregates, comparators, numbers (HINTS only)
     v
[3 Schema linking]      SBERT embeddings -> columns/tables, prune schema
     v
[4 Value linking]       rapidfuzz vs real DB values
     v
[5 Example retrieval]   top-4 similar solved examples (semantic search)
     v
[6 Prompt builder]      rules + pruned schema + hints + examples + question
     v
[7 LLM call]            Gemini returns SQL text (cached, temperature 0)
     v
[8 Validator]           SELECT-only check
     v
[9 Executor]            READ-ONLY SQLite connection
     v
   error? -> one retry: send error + failed SQL back to LLM, repeat 8-9
     v
Result table + all intermediate outputs shown in the UI
```

Key principle: the LLM only writes SQL text. It never touches the database. Our code validates the text and runs it.

Two run modes of the same pipeline:

- **full:** steps 1 to 9 as above (the real system).
- **baseline:** skips steps 1 to 5; prompt contains rules + FULL schema + question only (zero-shot). Used only for the comparison run in evaluation.

5. Repository Layout

```
text2sql/
  .env.example            # GEMINI_API_KEY=, GEMINI_MODEL=
  requirements.txt
  README.md
```

```
  config.py
  app.py                  # Streamlit UI
  db/
    create_db.py          # builds college.db
    schema_utils.py       # reads schema text
  nlp/
    preprocess.py         # analyse()
    extraction.py         # extract_hints()
    schema_linking.py     # link_schema()
    value_linking.py      # link_values()
    retrieval.py          # build_index(), retrieve_examples()
  llm/
    prompt_builder.py     # build_prompt()
    client.py             # generate_sql() with cache
  core/
    validator.py          # is_safe()
    executor.py           # run_query()
    pipeline.py           # answer()
  evaluation/
    evaluate.py
    metrics.py
  data/
    college.db
    examples.json         # 40 solved examples (training material for
prompts)
    test_questions.json   # 15 exam questions (NEVER used in prompts)
    example_embeddings.npy
    llm_cache.json
    results/
  tests/                  # pytest files, one per module
```

Add **init**.py files so packages import cleanly.

6. Configuration (config.py)

All tunable values MUST live here, not scattered in code.

Key	Default	Meaning
DB_PATH	data/college.db	database file
SCHEMA_LINK_THRESHOLD	0.40	min cosine similarity for a schema link

Key	Default	Meaning
VALUE_LINK_THRESHOLD	80	min rapidfuzz ratio (0-100) for a value link
MIN_FUZZY_LEN	4	tokens shorter than this must match exactly (case-insensitive)
TOP_K_EXAMPLES	4	retrieved examples per question
MAX_RETRIES	1	self-correction attempts
LLM_CALL_DELAY	1.0	seconds to sleep after each real (non-cached) LLM call
MAX_ROWS	500	rows fetched for display
EMBED_MODEL	all-MiniLM-L6-v2	sentence-transformers model

Thresholds are starting points. The human tunes them by inspecting linking behaviour (Section 12 rule on tuning).

7. Data Design

7.1 Database schema (SQLite)

```
CREATE TABLE students(
  id INTEGER PRIMARY KEY, name TEXT, branch TEXT,
  semester INTEGER, cgpa REAL, city TEXT);
CREATE TABLE courses(
  course_id INTEGER PRIMARY KEY, course_name TEXT,
  credits INTEGER, branch TEXT);
CREATE TABLE marks(
  student_id INTEGER, course_id INTEGER, marks INTEGER, attendance INTEGER,
  FOREIGN KEY(student_id) REFERENCES students(id),
  FOREIGN KEY(course_id) REFERENCES courses(course_id));
```

Relations: marks.student_id -> students.id; marks.course_id -> courses.course_id. A query needing students and courses MUST join through marks.

7.2 Seed data (create_db.py, random.seed(42), re-runnable)

- 60 students, ids 1 to 60. branch from CSE, ISE, ECE, MECH. semester from 3, 5, 7. cgpa random 6.00 to 9.80 (2 decimals). city from Bengaluru, Mysuru, Mangaluru, Hubballi, Chennai, Hyderabad. Names built from first names (Ravi, Priya, Arjun, Ananya, Karthik, Sneha, Rahul, Divya, Vikram, Meera, Suresh, Pooja, Aditya, Kavya, Manoj) and surnames (Sharma, Rao, Gowda, Nair, Reddy, Iyer, Patil, Shetty).
- 16 courses, 4 per branch, credits 3 or 4:
  - CSE: DBMS, Algorithms, Operating Systems, NLP
  - ISE: Data Mining, Computer Networks, Software Engineering, Cloud Computing
  - ECE: VLSI, Signals and Systems, Embedded Systems, Communication Theory
  - MECH: Thermodynamics, Fluid Mechanics, Machine Design, Robotics
- marks: every student gets a row for each of the 4 courses of their own branch. marks 30 to 100, attendance 55 to 100.

The seed data MUST contain the exact spellings Mangaluru and DBMS, because value linking is tested against them.

7.3 Example library (data/examples.json) - HUMAN-REVIEWED

40 solved pairs used as prompt examples. Fields per item: id (int), question (text), sql (text), tags (list of: count, filter, aggregate, group_by, order_limit, join). Cover all tags. The agent may DRAFT these, but the human reviews and runs every SQL. Every SQL MUST execute without error.

7.4 Test set (data/test_questions.json) - AUTHORED BY THE HUMAN

15 questions: 5 easy, 5 medium, 5 hard, with at least 5 deliberately messy (synonym like pupils, spelling like Mangalore, wrong case like dbms, hidden aggregate like typical marks). The agent MUST NOT write, edit or tune against these questions. The agent only builds the loader and validates the format.

Fields per item:

Field	Type	Meaning
id	int	1 to 15
level	text	easy, medium or hard

Field	Type	Meaning
messy	bool	true if it contains a synonym, misspelling or case issue
question	text	the natural-language question
gold_sql	text	correct SQL written by the human
gold_tables	list	tables the correct SQL needs
gold_columns	list	columns in the form table.column the question refers to
gold_values	list	pairs of column and stored value, e.g. students.city = Mangaluru

Integrity rule: no test question text may equal or closely match any example question (enforced in Section 8.5).

8. Module Specifications

8.1 nlp/preprocess.py

```
analyse(question: str) -> dict
```

Returns: tokens (list of str, original text), lemmas (lowercase lemmas excluding stop words and punctuation), pos (list of [token, POS tag]), noun_chunks (lowercase list), numbers (list of str).

Load the spaCy model once at module level. Empty input returns empty lists. Concepts demonstrated: tokenization, lowercasing, stop-word removal, lemmatization, POS tagging, noun-chunk extraction.

8.2 nlp/extraction.py

```
extract_hints(question: str) -> dict
```

Returns: aggregates (list of dicts with phrase and function), conditions (list of dicts with phrase, operator, value), top_n (int or None).

Trigger tables (case-insensitive, whole-word match, match the LONGEST phrase first so at least wins over least):

Phrases	Result
how many, number of, count	COUNT
average, mean, typical	AVG
total, sum	SUM
highest, maximum, most, best	MAX
lowest, minimum, least, worst	MIN
top N, top	TOP_N (capture N if a number follows)
more than, greater than, above, over, higher than	operator >
at least	operator >=
less than, fewer than, below, under, lower than	operator <
at most	operator <=
equal to, exactly	operator =
between X and Y	BETWEEN with two values

For comparators, capture the number (int or decimal) that immediately follows. The output is HINTS only. The module MUST NOT try to decide which column a number belongs to; the LLM decides that. Report this honestly in the project report.

8.3 nlp/schema_linking.py

```
link_schema(phrases: list[str], threshold: float) -> dict
```

Returns: columns (dict mapping phrase to a dict with column and score), tables (sorted list of linked tables), scores_log (every phrase with its top-3 candidates, for tuning).

Schema descriptions (embedded once at import with EMBED_MODEL, normalized vectors, cosine via dot product):

Key	Description text
table students	students pupils learners scholars people enrolled in the college
table courses	courses subjects papers modules taught
table marks	marks scores results grades obtained in exams per student per course
students.id	student id roll number identifier
students.name	student name full name person
students.branch	branch department stream discipline
students.semester	semester sem term year of study
students.cgpa	cgpa gpa grade point average overall academic performance
students.city	city town hometown place location native
courses.course_id	course id code identifier
courses.course_name	course name subject name paper title
courses.credits	credits credit points weightage
courses.branch	branch department offering the course
marks.marks	marks score points obtained exam result
marks.attendance	attendance presence classes attended percentage present

Candidate phrases = noun_chunks + lemmas from analyse(), deduplicated, skipping phrases shorter than 3 characters and purely numeric ones. A hit on a table description adds that table. A hit on a column description adds that column and its table. A phrase links only if its best score is at least the threshold.

Schema pruning rule (used by the pipeline): pruned tables = tables from schema links plus tables of any linked values. If both students and courses are present, ALWAYS add marks (bridge table). If the result is empty, fall back to ALL tables. The relations line is always included.

8.4 nlp/value_linking.py

```
link_values(tokens: list[str], threshold: int) -> list[dict]
```

Returns a list of dicts: phrase, column, value, score.

Categorical columns checked: students.branch, students.city, courses.course_name. Distinct values are loaded from the database at startup (never hard-coded).

Algorithm:

1. Build n-grams (n = 1 to 3) from lowercased original tokens.
2. Skip n-grams that are all stop words, purely numeric, or made only of schema words (students, marks, average, attendance, and the words in the Section 8.3 descriptions).
3. For n-grams shorter than MIN_FUZZY_LEN characters, require an exact case-insensitive match. This prevents false matches on short words.
4. Otherwise compare with rapidfuzz fuzz.ratio, case-insensitive, and accept scores at or above the threshold.
5. Keep the best column and value per phrase. When spans overlap, keep the longer span.

Concepts demonstrated: n-grams, edit distance, approximate string matching.

8.5 nlp/retrieval.py

```
build_index() -> None
retrieve_examples(question: str, k: int) -> list[dict]
```

build_index encodes all example questions, saves data/example_embeddings.npy, and runs the leak check: raise an error if any test question equals an example question (case-insensitive), and print a warning for any test/example pair with cosine similarity above 0.95. retrieve_examples encodes the question and returns the top-k examples (each with its similarity score) by cosine similarity. Rebuild the index automatically if the file is missing or the example count changed.

8.6 llm/prompt_builder.py

```
build_prompt(question, schema_text, hints, examples, error, previous_sql) ->
str
```

See Section 9 for the exact template. hints and examples may be empty (baseline). error and previous_sql are set only for the retry.

8.7 llm/client.py

```
generate_sql(prompt: str) -> str
```

- Use google-genai with temperature 0 for reproducibility (the agent MUST verify the correct SDK syntax from the installed version).
- Cache: key = sha256 of model name + prompt, stored in data/llm_cache.json. A cache hit makes no API call. A flag MUST allow bypassing the cache.
- On transient errors, retry up to 3 times with exponential backoff. After each real call, sleep LLM_CALL_DELAY seconds.
- Clean the output: remove code fences, remove a leading sql label, keep only the first statement, ensure one trailing semicolon.
- Never log the API key.

8.8 core/validator.py

```
is_safe(sql: str) -> tuple[bool, str]
```

Steps: strip SQL comments; blank out the contents of string literals (so a value like update inside quotes does not trigger a false block); lowercase; require the text to start with select; require exactly one statement (no semicolon except the final one); reject any whole-word occurrence of: drop, delete, update, insert, alter, create, replace, attach, detach, pragma, truncate, vacuum, reindex. Return (False, reason) when blocked.

8.9 core/executor.py

```
run_query(sql: str, max_rows: int) -> tuple[list, list]
```

The connection MUST be read-only, opened with a URI of the form file:PATH?mode=ro and uri=True. Return (column names, rows) using fetchmany(max_rows). Always close the connection. Let sqlite3.Error propagate to the caller.

8.10 core/pipeline.py

```
answer(question: str, mode: str) -> dict    # mode is full or baseline
```

Logic:

1. full mode: analyse -> extract_hints -> link_schema (on noun_chunks + lemmas) -> link_values (on tokens) -> retrieve_examples -> compute pruned schema text. baseline mode: full schema, no hints, no examples.
2. build_prompt -> generate_sql.
3. is_safe. If blocked, return with the error and DO NOT execute.
4. run_query. On sqlite3.Error, if retries remain (MAX_RETRIES), build a new prompt containing the error and the failed SQL, call the LLM again, and repeat steps 3 to 4. Never retry a blocked query. Never retry an empty (zero-row) result.

Returned dict fields: question, mode, nlp (analysis, hints, schema_links, value_links, retrieved_examples, pruned_tables), prompt, sql, columns, rows, error, retried (bool), llm_calls (int), timings (ms for NLP stage and LLM stage).

9. Prompt Template

The exact layout (sections in this order; omit the Hints and Examples sections in baseline mode and when empty):

```
You are an expert SQLite query writer.
Rules:
- Output ONLY one SQL query. No explanation. No markdown.
- Use only the tables and columns listed in the schema.
- Only write SELECT queries.
- Use JOINs and table aliases when more than one table is needed.
- Use the exact stored values shown in the hints when filtering text columns.

Database schema:
<pruned or full schema text, including categorical value lists and the
relations line>

Hints from NLP analysis (suggestions, verify against the question):
- Aggregation: ...
- Numeric conditions: ...
- Likely columns: ...
- Likely values: column = stored value

Similar solved examples:
Question: ...
SQL: ...
(repeat for the retrieved examples)
```

```
<only on retry>
Your previous SQL was: ...
It failed with this error: ...
Return only the corrected SQL.

Question: <user question>
SQL:
```

Schema text format (from db/schema_utils.py): one line per table in the form Table name(col TYPE, ...), then a comment line of possible values for every TEXT column with 8 or fewer distinct values, then the relations line.

10. Safety Design

Two independent layers: (1) the text validator of Section 8.8, and (2) the read-only database connection of Section 8.9. Both MUST be present, so that a missed pattern in layer 1 still cannot modify data. The UI MUST show a clear message when a query is blocked.

11. Error Handling Summary

Situation	Behaviour
LLM returns non-SQL text	validator blocks it; show the message
SQL has a syntax or column error	one retry with the error fed back
Query valid but zero rows	show no rows found; no retry
API rate limit or transient failure	backoff retries, then show a friendly error
Destructive SQL	blocked before execution

12. Evaluation Design (evaluation/evaluate.py)

Runs the 15 test questions through the pipeline in full mode, then again in baseline mode (about 15 LLM calls each, fewer when cached). Saves every record to data/results/.

Metrics:

Metric	Definition
Execution accuracy	Share of questions whose returned rows equal the rows of gold_sql. Report overall, by level, and messy vs non-messy
Table recall	Share of gold_tables present in the pruned tables
Column recall	Share of gold_columns present in the schema links
Value recall and precision	Recall: gold_values found among value links. Precision: share of predicted value links that are correct (counts false matches)
Cost	LLM calls, average latency, prompt length in characters

Row comparison rule: run gold_sql through the same executor. Round floats to 2 decimals and convert every cell to text. Compare as an unordered multiset of rows, except when gold_sql contains ORDER BY, in which case compare as an ordered list. Compare result rows, never SQL text.

Outputs: per-question JSON, summary.json, and a markdown results table with the full vs baseline comparison, plus an error list with one line per failure (question, predicted SQL, gold SQL, error text).

Integrity rules:

1. The test set is never used in prompts or examples.
2. Thresholds are tuned by inspecting linking behaviour (scores_log), NOT by editing prompts to fix specific test failures. If a prompt or rule change was made after seeing test results, record it in a CHANGELOG so the report can state it honestly.
3. All LLM results are cached; reruns reuse the cache unless bypassed.

13. Streamlit UI (app.py)

- Title and a text box for the question; an Ask button; a sidebar with example questions and a view of the schema.
- A mode switch (full or baseline) for demonstration.

- After asking, show in this order: the final SQL; the result table (pandas dataframe); a note if the retry fired; an expandable NLP Analysis section containing tokens and lemmas, POS tags, noun chunks, detected aggregates and conditions, schema links with scores, value links with scores, retrieved examples with similarity, pruned tables, and the full prompt.
- Blocked queries show a clear warning. Errors never show a raw stack trace.

14. Build Plan (phases, stop after each)

Phase	Tasks	Done when
1 Foundation	Environment and requirements; config.py; create_db.py; schema_utils.py; loaders for examples.json and test_questions.json (human supplies the content)	One test call to the LLM returns text; the database passes the Section 15 checks
2 Baseline	prompt_builder (basic), client.py with cache, validator.py, executor.py, retry in pipeline (baseline mode)	Command-line question returns an answer; unsafe SQL is blocked
3 NLP modules	preprocess, extraction, schema_linking, value_linking, retrieval, then integrate into the full pipeline	Each module passes its own tests, then the full pipeline runs on all 15 questions with prompts inspected by the human
4 Interface	app.py with the NLP analysis panel	Live demo works, including a messy question and a malicious one
5 Evaluation	evaluate.py, metrics.py, results tables, error list	summary.json and the results table exist
6 Documentation	README, architecture diagram, results write-up	Human can reproduce everything from the README

15. Acceptance Checks

The agent MUST run these and show the actual outputs. Where spaCy output is involved, report the real output for the human to judge instead of forcing a match.

Database: 60 students, 16 courses, 240 marks rows; the value Mangaluru exists in students.city; DBMS exists in courses.course_name.

analyse(): for the running example, the lemmas should include pupil, mangalore, dbms, attendance; noun_chunks should include pupils.

extract_hints(): the running example gives aggregate AVG and condition operator > with value 75. For at least 5 marks, the operator is >=, not >. For top 5 students, top_n is 5.

link_schema(): pupils links to the students table; attendance links to marks.attendance; grades links to students.cgpa or marks.marks. Print scores_log and let the human tune the threshold.

link_values(): Mangalore links to students.city = Mangaluru; dbms links to courses.course_name = DBMS; average and marks produce NO value link; cse links to students.branch = CSE only as an exact match.

retrieve_examples(): for 5 sample questions, the human reads the 4 retrieved examples and confirms they are relevant. The leak check passes.

Validator: blocked: DROP TABLE students; and SELECT * FROM students; DELETE FROM students; and UPDATE students SET cgpa = 10. Allowed: SELECT name FROM students WHERE cgpa > 9; and a SELECT whose string literal contains the word update.

Executor: a normal SELECT runs; an attempted write on the read-only connection raises an error.

Pipeline: a deliberately broken query triggers exactly one retry; a zero-row result triggers none; llm_calls is reported correctly.

16. Rules for the Agent

1. Follow the phases in order and stop for confirmation after each.
2. Never touch, generate or tune against data/test_questions.json.
3. Never hard-code answers, test questions or gold SQL into any module.
4. NLP outputs are hints to the LLM, never final truth.
5. Keep thresholds in config.py only.

6. Do not add features, libraries or files not listed here without asking.
7. Write a pytest file for each module, and add short comments explaining the NLP concept in each file (the human must be able to explain them).
8. Keep a CHANGELOG.md of every decision and any change made after seeing evaluation results.
9. Never commit or print secrets; provide .env.example only.
10. When something fails, report the real error and your proposed fix instead of silently working around it.

17. Known Limitations (to state in the report)

- The test set is small (15 questions), so results show trends, not precise accuracy.
- Rule-based extraction detects that a condition exists but not always which column it applies to.
- Schema descriptions are written by hand for one small database, so linking quality depends on them.
- One database and one language (English); ambiguous questions like top students are answered with the LLM's best guess.
- Free LLM tiers can be slow or rate limited, so results are cached.

18. Future Work

Fine-tuning T5 on WikiSQL as a comparison, a query-type classifier, clarification questions for ambiguous input, automatic schema description generation, and support for larger multi-table schemas.